Overview

AI engineers integrate model services, feature stores, and external providers through HTTP APIs. Strong API fundamentals prevent brittle integrations and silent data corruption.

HTTP Basics

HTTP request includes method, URL, headers, and optional body.

Core methods: - `GET`: retrieve resource. - `POST`: create resource or trigger action. - `PUT`: full update/replace. - `PATCH`: partial update. - `DELETE`: remove resource.

Status codes families: - `2xx`: success. - `4xx`: client/request issues. - `5xx`: server-side failure.

JSON Basics

JSON supports objects, arrays, numbers, strings, booleans, and null.

API integration risks: - Missing fields. - Unexpected nulls. - Type mismatches (`"42"` vs `42`).

Always validate schema before downstream model usage.

REST API Patterns

REST principles: - Resource-oriented endpoints (`/predictions`, `/users/{id}`). - Stateless requests. - Standard HTTP semantics.

Common patterns: - Pagination (`page`, `limit`). - Filtering (`?status=active`). - Versioning (`/v1/predict`).

AI Engineering Use Cases

- Calling LLM/embedding APIs.
- Pulling external features from internal services.
- Serving model predictions through `/predict` endpoint.
- Logging inference traces for monitoring.

Common Pitfalls

- Not setting request timeout.
- Ignoring non-200 status codes.
- Blindly calling `.json()` on non-JSON responses.
- Missing retry/backoff for transient failures.

Business Case Studies & Exceptions

Case 1: Timeout Cascade in Recommendation API

Upstream feature API slowed down; inference API threads blocked waiting indefinitely.

Fix pattern: - Short request timeout. - Retry with capped exponential backoff. - Circuit breaker fallback to cached features.

Case 2: Schema Drift in Vendor API

Field renamed from `score` to `risk_score`, breaking downstream parser.

Fix pattern: - Response schema validation. - Alert on schema mismatch. - Backward-compatible parsing layer.

Interview Questions & Answers

1. **Q: How call REST API from Python safely?**
   A: Use `requests` with timeout, check `status_code`, handle exceptions, then parse and validate JSON.
2. **Q: Meaning of 200, 404, 500?**
   A: 200 success, 404 resource not found, 500 server-side error.
3. **Q: Why check status before `.json()`?**
   A: Error responses may be HTML/text or different schema, causing parse/runtime failures.
4. **Q: GET vs POST for model inference?**
   A: POST is typical for inference payload bodies and non-idempotent processing semantics.
5. **Q: What is idempotency?**
   A: Repeating request yields same effect; important for retries and consistency.
6. **Q: Why add retries carefully?**
   A: Retries improve resilience for transient errors but can amplify load if uncontrolled.
7. **Q: What metadata to log on API failure?**
   A: Endpoint, method, status code, latency, request ID, exception class, and retry count.
8. **Q: How secure API keys?**
   A: Store in environment/secret manager, never hardcode in repo.
9. **Q: What is schema validation role in AI systems?**
   A: Prevent malformed external data from poisoning feature pipelines or inference responses.
10. **Q: How design simple prediction endpoint?**
    A: `POST /v1/predict` with validated JSON input, structured output, error codes, and traceable logs.